Sankhya — Honest Assessment

What works, what does not, and what would fix it

2 September 2026

Contents

1. Scorecard

Against **HiGHS**, the realistic open-source bar, on this project's own assessment. Every row has evidence behind it in `docs/RESULTS.md`.

component	score	why
MPS reader	**95**	at parity with HiGHS on all 88 Netlib models, and 1.4–1.5× faster
Infrastructure	**78**	14 test suites, CI on every push gating on correctness. No packaging
GPU	**65**	3.14×–12.07×, verified on hardware with contract tests passing
First-order LP	**64**	strongest algorithmic piece; threaded, though only 1.41×
Simplex	**58**	82/88 Netlib correct, **none wrong**; takes a basis from the first-order method
Presolve	**50**	12 reductions against PaPILO's ~25
QP	**50**	OSQP's core; no AMD ordering, five instances fail
MILP	**40**	**the weak leg**

Weighted: roughly 65 out of 100. It was 50 at the start of this work.

The largest single contributor to that move is not speed. It is that six wrong answers became zero.

2. Strengths

2.1 Correctness, and the machinery that keeps it

Zero wrong answers across all 88 Netlib instances with published optima. 82 reach the optimum; the other six fail *visibly* — a time limit, an iteration limit, or a numerical error the caller can see. At the start of this work six of them returned confident wrong numbers, including one that reported `optimal` at a point missing the constraints by 1.8×10^9.

CI gates it. Every push builds on Linux and macOS, runs the fourteen suites, and then runs all 88 instances against published optima. A wrong answer turns the build red. In its first two runs it found that the project had never compiled on Linux, and that the fetch script was silently undoing manual corrections to the reference table.

Six layers of testing, each of which exists because something got through the one below it: unit tests, verification against published optima, cross-validation against HiGHS, self-checks inside the solver, a debug-solution tracker that names the prune which discarded a known answer, and invariant checks that need no reference at all.

The reference data itself was wrong and that was found. Eight optimal values in the Netlib README are incorrect — it handles the objective row's constant differently from every solver that reads the file, and `e226` is off by exactly 7.113, which is that instance's own constant. Corrected against HiGHS, in a separate file so re-fetching cannot undo them.

2.2 Built from scratch, and it can be checked

16,152 lines with no dependency outside the standard library. No HiGHS, no OR-Tools, no SciPy underneath. HiGHS appears in the repository exactly once: as the thing being benchmarked against.

The reader, presolve, LU factorisation, simplex, branch-and-cut tree, ADMM loop, CUDA kernels and thread pool are all in `src/`.

2.3 Two solvers that talk to each other

Crossover is the piece of genuine engineering here rather than a paper implemented. The first-order method reaches a *point* quickly and a vertex never; the simplex reaches a *vertex* with a certificate and pays for the walk. Crossover reads a basis off the first-order point and starts the simplex there.

- roughly **half the simplex pivots** across Netlib — `degen3` 89,640 → 25,027
- handed its own optimum, it returns in **zero** iterations
- **three instances go from no answer at all to the published optimum**: `modszk1`, `stocfor2`, `woodw`

It cannot cost an answer: a seeded solve that does not reach an optimum falls back to the cold one.

2.4 The method is precisely identified

Proposition 3.1 of arXiv:2509.23903 establishes that reflected restarted Halpern PDHG at $\gamma = 1$ **is** the Halpern Peaceman–Rachford method. Our default reflection is 1.0, so this is an HPR method — a checkable statement about what the code is, not a claim about how good it is.

2.5 Threading that does not change the answer

Bit-identical to serial at every thread count. Not "reproducible at a fixed thread count", which is what production solvers actually guarantee, and not "agrees to 10^{-13}" — the same bits. Verified three ways, including 88 of 88 instances identical on objective, iteration count and status.

2.6 GPU verified on hardware

3.14×–12.07× on the solve, CPU/GPU agreement at or below machine precision, and identical iteration counts on every Netlib instance tried. The verification run found a real device-only bug (see `Sankhya_GPU_Report.pdf` §4).

2.7 Discipline that is visible in the source

Every tuned constant carries its measurement. The refactorisation trigger, the polish schedule, the unsafe-pivot fraction, the crossover seed budget — each has the sweep that set it written beside it.

Failures are kept, not deleted. Interior point, hypersparsity, Forrest–Tomlin, coefficient tightening, parallel column merging, the Harris ratio test, two QP fixes — all implemented or measured, all rejected, all recorded with the numbers that rejected them.

3. Weaknesses

3.1 MILP is genuinely behind

On 70 MIPLIB instances at a 15-second limit: **46 end with a feasible solution, 7 prove optimality, 24 find nothing at all.**

Head to head with HiGHS on 40 of them, single threaded, same limit:

	this solver	HiGHS
found any solution	27	**37**
proved optimality	4	**13**
closer, where both found one	2	**15**

That is the honest gap and it is the largest one. Missing: conflict analysis, restarts, clique tables, symmetry detection, node presolve, and a cut pool with ageing rather than one growing matrix.

3.2 QP has five failures and no owner

35 of the 40 smallest Maros–Meszaros instances solve. The five that fail are one family — `PRIMALC1/2/5/8` — whose `DUALC` counterparts all solve. The cause is diagnosed: adaptive ρ collapses and the gate that stops it oscillating also stops it recovering. **Three fixes have been tried and all three failed.** No AMD ordering for the $LDL^\top$.

3.3 Threading is 1.41×, and only on one component

The simplex and the branch-and-bound tree are still serial. The first-order method is threaded and peaks at 1.41× on five threads, then *loses* from eight. The cause is architectural — a sparse matrix-vector product is memory-bandwidth bound, and threads contend for the bus rather than helping — but the number is 1.41×, not 6×.

3.4 Six Netlib instances still do not finish

`cycle` and `wood1p` end in numerical errors. `d6cube` and `scsd8` hit iteration limits — and `scsd8` reaches **904.9999999 against a published 904.99999993**, correct to ten digits, without being able to prove it. `dfl001` and `pilot87` run out of time.

3.5 Scale is untested

The problem statement mentions "thousands to millions of variables". The largest instance tested is `dfl001` at roughly 6,000 rows, and it times out. Nothing here demonstrates the scale the statement describes.

3.6 Presolve can make the first-order method worse

Over the full Netlib set with presolve on, three instances stop converging that converged without it — `agg2` and `agg3` from 6,400 and 16,960 iterations to the 1,000,000 cap. Geometric mean is still 1.38× faster, but the worst case is 0.01×.

3.7 Smaller gaps

- **12 presolve reductions** against PaPILO's ~25; no probing, no clique extraction.
- **No packaging** — no install target, no C API, no Python binding.
- **No MIQP, NLP or MINLP**, which the statement mentions as extensions.
- **Missing simplex techniques** — Forrest–Tomlin, bound-flipping ratio test, hypersparsity. All three were measured and rejected on this instance set, but they are absent.
- **Indicator constraints** are rejected with a clear message rather than parsed. One MIPLIB instance in 103 uses them.

4. What would improve it, in order

4.1 Immediate

	what	why it matters
1	**Give the refinery model more integer structure**	Done once — `refinery_milp.mps` now exists at 1,500 rows and 120 integers, and the result is 2.5% behind HiGHS on the objective. More of it would make the MILP leg's home instance harder and more representative
2	**Report against LPfeas more widely**	Four of eight solved so far; the remaining instances would place this against cuPDLPx and OR-Tools PDLP on the benchmark those codes are actually scored on

4.2 Short term — the MILP leg, which is where the points are

	what	expected effect
3	**Conflict analysis**	When a node is proved infeasible, turn the reason into a constraint that prunes elsewhere. The 24 instances finding nothing are where this lands
4	**Restarts**	Throw the tree away once enough is known about the variables and rebuild with better branching order
5	**Cut pool with ageing**	A cut currently enters the matrix and stays forever, making every node below it more expensive
6	**Probing in presolve**	Fix a binary to 0 and to 1, propagate each, keep what both agree on. Usually the highest-value MILP reduction and it is absent
7	**Clique extraction**	Feeds both the cut separator and probing

4.3 Short term — the other components

	what	expected effect
8	**QP: hard restart of ρ**	Reset to the initial value and refactorise. Structurally different from the three fixes that failed, and untried
9	**QP: AMD ordering**	The $LDL^\top$ currently uses the natural order; measure the fill first
10	**Scale test on large instances**	The "millions of variables" claim is untested and should either be demonstrated or dropped

4.4 Longer

	what
11	Deterministic parallel branch-and-bound — the tree is the largest unparallelised component
12	Packaging: install target, C API, Python binding
13	Forrest–Tomlin updates, if a profile ever shows the factorisation as hot
14	MIQP, as the statement's named extension

5. The one thing worth taking away

The algorithms were not the hard part. They are in papers and they work.

What took the time was finding out **when the solver was quietly wrong** — and building the things that make that visible.

A slow solver tells you it is slow. A wrong one tells you nothing: it returns a confident number with a matching dual bound and no complaint. This one reported `optimal` at a point missing the constraints by 1.8×10^9. It declared a bounded model unbounded. It returned a proved optimum that was 60.8% wrong. On the GPU it returned one matrix's answer for a different matrix, and the verification harness reported "GPU backend is correct" while the test that caught it was red.

None of those was found by a test written for it. They were found by checking answers against published values, by two independent methods disagreeing about the same problem, and by a third solver settling which of the two was right.

Every self-check in this codebase exists because something got through.